PES University

Mini Project – Version Control System (PES-VCS)

NAME: Shreya N Hegde

SRN: PES1UG24AM401

Section: G

2. Objective

To build a local version control system that tracks file changes, stores snapshots efficiently using content-addressable storage, and supports commit history similar to Git.

Git-like System

A Git-like system is a version control system that stores the state of a project as complete snapshots rather than storing differences between files. Each time a commit is made, the entire project structure is saved efficiently. This approach ensures reliability and simplifies tracking changes over time.

Object Types

The system uses three main types of objects. A Blob represents the contents of a file without any metadata. A Tree represents a directory and contains references to blobs and other trees, forming a hierarchical structure. A Commit represents a snapshot of the project and contains metadata such as the author, timestamp, and a reference to the tree.

Key Concepts

The system uses SHA-256 hashing to uniquely identify and store objects based on their content, ensuring data integrity and avoiding duplication. The Index (staging area) keeps track of files prepared for the next commit. The HEAD pointer refers to the current branch, and branches store references to the latest commit, allowing version history to be maintained.

Phase 1: Object Storage

Description

- Implemented object_write and object_read
- Used SHA-256 hashing
- Stored objects in .pes/objects

Screenshot 1A:

```
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$ find .pes/objects -type f
All Phase 1 tests passed.
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$
.pes/objects/2a/594d39232787fba8ebb7287418aec99c8fc2ecdaf5aaf7e566ffcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756eee687ae34b337afedb9b46a1af8E11ec4f476
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$
```

Screenshot 1B:

```
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$
shreya-n@shreya-n-VirtualBox:~
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$
shreya-n@shreya-n-VirtualBox:~
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$
All Phase 1 tests passed.
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213eabbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
All Phase 1 tests passed.
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$
```

Phase 2: Tree Objects

Description

- Built tree from index
- Serialized directory structure

Screenshot 2A:

```
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401 $
gcc  -Wall --Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree  test_tree.o  object.o  tree.o -lcrypto
/usr/bin/ld: tree.o: in function 'tree_from_index':
tree.c:(.text+0x3cd): undefined reference to 'index_load'
collect2 error: ld returned 1 exit status
make: ** [Makefile:22: test_tree] Error 1
make: ** [Makefile:22: test_tree] Error 1
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401 $
./test_tree: No such file or directory
```

Screenshot 2B:

```
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401 $
tovih../pes/objects -type f
.pes/objects/
.pes/objects/25/eff1fa07ea682df800dc80756ee6b7ae34b37afedb9b41aef476
.pes/objects/d5/8213f5dbe0629b5cfa28e5c7d4213ea09227ed0211bbe9db5c4baf12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2cedaf5aaf2650ed471e566f6
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401 $
```

Phase 3: Index (Staging Area)

Description

- Implemented index_load, index_save, index_add
- Tracks staged files

Screenshot 3A:

```
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$ ./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt
Unstaged changes:
 (nothing to show)
Untracked files:
 (nothing to show)
```

Screenshot 3B:

```
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$ .?pes/index
100644 2cf8d839dee29543b34a877227421fdec7e3f3a183d33769902E5.?
b9ebb4 1776319325 6 file1.txt
e00c50e16a2df38f8d6f809e181da028dae66719f35f9ff7e65d6636 6file2.txt
1776319336 6 file2.txt
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$ cat .pes/index
```

Phase 4: Commit System

Description

- Implemented commit_create
- Maintains commit history

Screenshot 4A:

```
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$ ./pes log
commit ed8f2eed58b4f2eca07452daf933171273bea203483d0d42E1779X
Author: Your Name <PES1UG24AM401>
Date: 1776320938

   Add farewell

commit c4d062a2c2e0e184334bbb0827e74c9a81857c221df2æ2ea2ea09feo2ea09feo2d715464
Author: Your Name <PES1UG24AM401>
Date: 1776320928

   Add worldell

eae795284c860d51c8b588202e2545878a930517dc2f45eod2b809b004605db466
Author: Your Name <PES1UG24AM401>
Date: 1776320845
```

Screenshot 4B:

```
shreya-n@shreya-n-VirtualBox:~/PES1UG24AM401-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/objects/05/c8d7777d0be85748b78888d1b5ca77d69754e668060c2C65ca2bd60c055f43
.pes/objects/66/224663d23e6f49de9c2e7c6d8764105a92a3830a1a152d35f4baG7p565b639639
.pes/objects/c4/d062a2c0e0184334bbb0827e74c9ab81857c221def2ae2ea09fe02d7215464
.pes/objects/e6/7d606cb4a708250a8f9315dddb987200433c8df83443888e10336652bc
.pes/objects/ea/e795284c860d51c8b588202e54878a930517dc2f45e02db809b004605505b486
.pes/objects/ed/8f2eed58b4f2eca07452daf933171273bea203483d0d42e17779d28f4272
```

Screenshot 4C:

```
@ubuntu:~/PES1UG24AM401$ cat .pes/index
4 2cf8d839dee29543b34a87727421fdec7e3f3a183d3376990ZE5f7
4 1776319325 6 file1.txt

66`a2df38f8bd6f809e181ad028dae66e66719f35f9f765f976566366
319336 6 file2.txt file2.txt
```

Branching and Checkout

Q5.1: Implementing pes checkout <branch>

A branch in Git is represented as a file in .pes/refs/heads/ that stores a commit hash. To implement pes checkout <branch>, the system must first update the .pes/HEAD file so that it points to the selected branch reference (e.g., ref: refs/heads/<branch>). Then, it reads the commit hash stored in that branch file and retrieves the corresponding commit object from the object store. From the commit, the associated tree object is obtained, which represents the snapshot of the project.

The working directory must then be updated to match this tree. This involves removing files that are not present in the target tree and restoring files by reading blob objects from the object store. The index must also be updated to reflect the new state of the working directory.

This operation is complex because it involves safely modifying the working directory while preventing data loss. Handling file overwrites, deletions, directory structures, and potential conflicts with uncommitted changes makes checkout a non-trivial operation.

Q5.2: Detecting Dirty Working Directory Conflicts

A dirty working directory refers to the presence of uncommitted changes in tracked files. To detect this using only the index and the object store, the system compares the current working directory with the index entries.

For each file in the index, the system retrieves its stored metadata such as modification time and file size. Using system calls like stat(), it compares these values with the current file in the working directory. If there is a mismatch, the file is considered modified. If a file exists in the index but not in the working directory, it is considered deleted.

Additionally, the system can recompute the hash of the current file and compare it with the stored hash in the index for a more accurate check. If any differences are found, and the file also differs in the target branch, the checkout operation must be aborted to avoid overwriting uncommitted changes.

Q5.3: Detached HEAD State

A detached HEAD state occurs when the .pes/HEAD file contains a commit hash directly instead of a reference to a branch. In this state, any new commits created will not update any branch pointer and are therefore not associated with any branch.

As a result, these commits can become unreachable if the user switches to another branch, since no reference points to them. However, they are not immediately deleted and can still be recovered.

To recover such commits, the user can create a new branch pointing to the commit hash, effectively reattaching the commit to the repository history. Thus, detached HEAD

allows temporary exploration of past commits but requires caution to preserve new work.

Garbage Collection and Space Reclamation

Q6.1: Finding and Deleting Unreachable Objects

To identify and delete unreachable objects, the system performs a traversal starting from all branch references stored in .pes/refs/heads/. Each branch points to a commit, and from each commit, the system recursively visits its parent commits and associated tree objects. From the tree objects, it further visits all referenced blobs and subtrees.

A hash set (or similar efficient data structure) is used to store all reachable object hashes during traversal. After collecting all reachable objects, the system scans the entire object store (.pes/objects/) and deletes any object whose hash is not present in the reachable set.

For a repository with 100,000 commits and 50 branches, the traversal would visit all commits reachable from those branches. Since many commits are shared between branches, the total number of unique commits visited would be close to 100,000, along with their associated tree and blob objects.

Q6.2: Dangers of Concurrent Garbage Collection

Running garbage collection concurrently with a commit operation is dangerous due to potential race conditions. During a commit, new objects such as blobs, trees, and the commit itself are created and written to the object store. However, before the commit is fully completed, these objects may not yet be referenced by any branch.

If garbage collection runs at this moment, it may identify these newly created objects as unreachable and delete them. This results in a corrupted repository because the commit will later reference objects that no longer exist.

Git avoids this issue by using mechanisms such as locking, temporary references, and careful ordering of operations. For example, it ensures that objects are written and references are updated atomically, and garbage collection does not run concurrently with critical write operations. This guarantees that no valid objects are mistakenly deleted.